

Variáveis e controle de fluxo

Programação Imperativa

Prof. Pedro Rocha Horchulhack

Contato: p.horchulhack@pucpr.br

Agenda

1. Ambiente de desenvolvimento
2. Linguagem Rust
3. Variáveis e tipagem
4. Estruturas de controle de fluxo
5. Atividade

Agenda

- 1. Ambiente de desenvolvimento**
2. Linguagem Rust
3. Variáveis e tipagem
4. Estruturas de controle de fluxo
5. Atividade

Ambiente de desenvolvimento

- Como o Rust é uma linguagem compilada, precisamos baixar seu compilador: `rustc`
- O compilador é o responsável por converter nosso código em Rust para um objeto executável na nossa máquina
- Baixar através daqui: <https://rust-lang.org/tools/install>

Ambiente de desenvolvimento

Instalação

Linux/MacOS

Abrir o terminal e executar o comando `curl -proto 'https' -tlsv1.2 -sSf https://sh.rustup.rs | sh`

Windows

Baixar o Visual Studio C++ Build Tools e o `rustup-init.exe` (64-bit). Os binários de instalação estão no link do slide anterior!

Ambiente de desenvolvimento

O procedimento instalará as seguintes ferramentas:

- `rustc`: Compilador do rust
- `cargo`: Gerenciador de pacotes e projetos em Rust
- `rustup`: Instalador do rust

Ambiente de desenvolvimento

- Estaremos utilizando a IDE RustRover, da JetBrains
- Também é possível integrar com o Visual Studio Code (VSCode)
- Acesse e baixe por este link: <https://www.jetbrains.com/rust/download/>



Agenda

1. Ambiente de desenvolvimento
- 2. Linguagem Rust**
3. Variáveis e tipagem
4. Estruturas de controle de fluxo
5. Atividade

Linguagem Rust

Hello World

Vamos fazer nosso primeiro programa em Rust, compilá-lo e executá-lo

1. Criar um arquivo chamado `main.rs`
2. Inserir como conteúdo:

```
fn main() {  
    println!("Hello, world!");  
}
```

3. Compilar o arquivo com o `rustc`
`rustc main.rs -o primeiro-programa-rust`

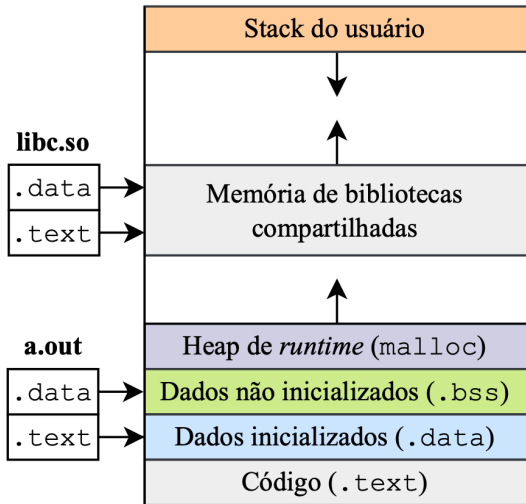
Linguagem Rust

Anatomia de um binário executável

- Todo programa desenvolvido depende de um ponto de início (*entrypoint*)
- Um *entrypoint* é uma especificação para o compilador da linguagem em questão, indicando onde o código inicia
- Por padrão, as linguagens e os compiladores definem os *entrypoints* como a função `main()`

Linguagem Rust

Anatomia de um binário executável



Linguagem Rust

Criação de um projeto

- Utilizamos o cargo para criação e gestão dos projetos em Rust
 - Build do código
 - Adicionar/remover pacotes terceiros
 - Testes unitários e/ou de benchmark
- Podemos criar um novo projeto com o comando “cargo new <nome do projeto>”

Linguagem Rust

Criação de um projeto

- Crie um projeto chamado “aula01”
- Abra o diretório no RustRover ou no VSCode

Estrutura do projeto:

```
aula01/  
├── Cargo.toml  
└── src/  
    └── main.rs
```

Linguagem Rust

Criação de um projeto

```
aula01/  
├── Cargo.toml  
└── src/  
    └── main.rs
```

- `Cargo.toml`: Informações do projeto (nome, autor, versão, dependências, ...)
- `src/`: Todos os códigos fonte do projeto devem ser criados neste diretório. A existência do arquivo “`main.rs`” é **obrigatória**.

Linguagem Rust

Criação de um projeto

- Para utilizarmos o cargo, temos dois principais comandos:
 - `cargo build`: Compila o código fonte e armazena no diretório “`target/debug/<nome do projeto>`”;
 - `cargo run`: Compila o código fonte e executa-o imediatamente;
 - `cargo add <nome pacote>`: Adiciona um pacote ao projeto.

Linguagem Rust

Um programa simples

Vamos definir um simples programa em Rust: Adivinhação de números.

```
1 use std::io;
2
3 fn main() {
4     println!("Jogo da adivinhação");
5
6     println!("Por favor, digite um número: ");
7
8     let mut numero: String = String::new();
9
10    io::stdin()
11        .read_line(&mut numero)
12        .expect("Erro ao realizar leitura");
13
14    println!("Você escolheu o número: {}!", numero.trim());
15 }
```


Linguagem Rust

Um programa simples

O que está acontecendo no exemplo anterior?

- **Linha 1:** Estamos utilizando (`use`), da biblioteca padrão do Rust (`std`), outra biblioteca para tratar entrada e saída (`io`).
- **Linha 3:** Iniciamos o *entrypoint* do código, sendo uma função (`fn main()`)
- **Linhas 4 e 6:** Macro para imprimir na tela a *String* delimitada por aspas duplas
- **Linha 8:** Criamos uma variável **mutável**, do tipo *String*, para armazenar o valor que recebemos por parâmetro

```
1  use std::io;
3  fn main() {
4      println!("Jogo da
    ↪   adivinhação");
5
6      println!("Por favor, digite
    ↪   um número: ");
8
    let mut numero: String =
    ↪   String::new();
```

Linguagem Rust

Um programa simples (continuação)

O que está acontecendo no exemplo anterior?

- **Linha 10:** Estamos acessando a função `stdin()` do módulo `io`, que é responsável pela entrada padrão (teclado) do sistema
- **Linha 11:** Chamamos a função `read_line()`, onde o seu parâmetro é uma **referência mutável** à variável `numero`
- **Linha 12:** Função específica de dois tipos: `Result` e `Option`. É utilizado para, em caso de erro, mostrar a mensagem fornecida como parâmetro.

```
10      io::stdin()  
11          .read_line(&mut numero)  
12          .expect("Erro ao realizar  
    ↪      leitura");
```

Linguagem Rust

Um programa simples (continuação)

O que está acontecendo no exemplo anterior?

- **Linha 14:** Macro para imprimir na tela a *String* delimitada por aspas duplas. A função `.trim()` é acessível para tipos `String`, responsável por remover os caracteres `\n` e `\t`. 14

```
println!("Você escolheu o  
↪ número: {}",  
↪ numero.trim());
```

Agenda

1. Ambiente de desenvolvimento
2. Linguagem Rust
- 3. Variáveis e tipagem**
4. Estruturas de controle de fluxo
5. Atividade

Variáveis e tipagem

Variáveis

- Por padrão, as variáveis na linguagem Rust são **imutáveis**, declaradas da seguinte maneira:

```
let x = 5;
```

Variáveis e tipagem

Variáveis

Devido as variáveis serem imutáveis, se tentarmos executar o seguinte código Rust, ele não irá compilar:

```
fn main() {  
    let x = 5;  
  
    println!("O valor de x é {x}");  
  
    x = 5 * 2;  
    println!("Agora o valor de x é {x}");  
}
```

Prática

Implementem o trecho acima e discutam sobre o resultado obtido. Qual foi o erro?

Variáveis e tipagem

Variáveis

E quando quisermos alterar o valor de uma variável, o que fazemos?

```
let mut x = 5;
```

Variáveis e tipagem

Tipagem

- Rust é uma linguagem fortemente tipada
- Não podemos alterar os tipos das variáveis após definirmos o seu tipo ou após a **inferência**

Variáveis e tipagem

Tipagem

```
fn main() {  
    let x = 5;  
    let y = 3.14;  
    let z = "ola";  
}
```

Discuta com os colegas

Qual o tipo das variáveis acima?

Discuta com os colegas

Como o compilador do Rust sabe qual tipo de cada variável declarada?

Variáveis e tipagem

Tipagem

Prática

Execute o trecho de código abaixo e discuta o resultado com os colegas.

```
fn main() {  
    let mut x = 5; // Inferência de tipo  
    let y: i32 = 11; // Declaração do tipo  
  
    x = "Olá!";  
}
```

Variáveis e tipagem

Tipos primitivos

- Lidamos com dados a todo momento como, por exemplo:
 - Nomes
 - Datas
 - Dias de aniversário
 - Velocidade
 - Comprimento
- Para a construção de algoritmos, temos quatro tipos básicos de dados primitivos, ou seja, o tipo de dado mais básico (FORBELLONE, EBERSPÄCHER, 2022):
 - Inteiro
 - Real
 - Caracter
 - Lógico

Variáveis e tipagem

Tipos primitivos – Inteiros

Tamanho (bits)	Assinado	Não-assinado
8	i8	u8
16	i16	u16
32	i32	u32
64	i64	u64
128	i128	u128
arch	isize	usize

Variáveis e tipagem

Tipos primitivos – Inteiros

- O que significa esse tamanho?
- Assumem tamanhos pré-definidos por conta da memória
- Quanto espaço a variável ocupará

Tamanho (bits)	Assinado	Não-assinado
8	i8	u8
16	i16	u16
32	i32	u32
64	i64	u64
128	i128	u128
arch	isize	usize

Variáveis e tipagem

Tipos primitivos – Inteiros

- Como podemos saber o tamanho mínimo e máximo de cada tipo de variável assinada?
- Tamanho mínimo: $-2^{(n-1)}$
- Tamanho máximo: $2^{(n-1)} - 1$
- n : tamanho da variável

Tamanho (bits)	Assinado	Não-assinado
8	i8	u8
16	i16	u16
32	i32	u32
64	i64	u64
128	i128	u128
arch	isize	usize

Variáveis e tipagem

Tipos primitivos – Reais

- Dois tamanhos de tipo de variável de ponto flutuante
- Seguem o padrão IEEE-754
- Por padrão, em Rust variáveis de ponto flutuante são declaradas como `f64` se não especificarmos diretamente o tipo

Tamanho (bits)	Tipo
32	<code>f32</code>
64	<code>f64</code>

Variáveis e tipagem

Tipos primitivos – Caracter

O tipo caracter (char) possui o tamanho de 4 *bytes* de tamanho (32-bits)

```
fn main() {  
    let c = 'z';           // Inferência de tipo  
    let z: char = 'A';     // Declaração de tipo  
}
```


Variáveis e tipagem

Tipos primitivos – Lógico

O tipo booleano pode assumir dois valores: verdadeiro ou falso (true ou false)

```
fn main() {  
    let porta_aberta = false;           // Inferência de tipo  
    let lampada_acesa: bool = true;    // Declaração do tipo  
}
```

Variáveis e tipagem

Leitura de dados do teclado

- Um sistema sempre coletará dados do teclado através de um *handler* para interagir diretamente com o terminal (*standard input*, `stdin`);
- Em linguagens imperativas precisamos declarar esse handler manualmente;
- Ele é o responsável por ler a linha do terminal e armazenar o resultado em alguma variável;

Variáveis e tipagem

Leitura de dados do teclado

```
use std::io;

fn main() {
    let mut buffer = String::new();
    io::stdin().read_line(&mut buffer).expect("Erro ao ler
    ↪ linha");
}
```

- Esse trecho de código nos mostra como fazer a leitura de dados do teclado por meio da entrada padrão (*standard input*, `stdin`).
- Aqui criamos uma região de memória arbitrária (comumente chamada de *buffer*) para armazenar o resultado coletado do teclado.

Variáveis e tipagem

Leitura de dados do teclado

Como é feita a conversão de tipos nesse caso?

```
use std::io;

fn main() {
    let mut buffer = String::new();
    io::stdin().read_line(&mut buffer).expect("Erro ao ler
    ↪ linha");

    buffer = buffer.trim().to_string();

    println!("{:.2}", buffer.parse::<f32>().unwrap());
}
```

Variáveis e tipagem

Exercícios

1. Escreva um algoritmo em Rust para calcular a idade de alguém, sabendo seu ano de nascimento
2. Escreva um algoritmo em Rust para calcular o valor, em reais, que deve ser pago para um cliente de uma locadora de carros. Sabe-se que:
 - O valor da locação para cada carro é 100.00 reais;
 - O cliente pode locar um único carro por vários dias;
3. Leia a temperatura do teclado em *Celsius* e imprima o equivalente em *Fahrenheit*

$$C = (F - 32) \frac{5}{9} \quad (1)$$

Agenda

1. Ambiente de desenvolvimento
2. Linguagem Rust
3. Variáveis e tipagem
- 4. Estruturas de controle de fluxo**
5. Atividade

Estruturas de controle de fluxo

Expressões lógicas

- Expressões cujos operadores são lógicos ou relacionais e cujos operandos são relações ou variáveis ou constantes do tipo lógico
- Operadores lógicos
 - E, OU e NÃO
- Como representamos em Rust?
 - E: `&&`
 - OU: `||`
 - NÃO: `!`

Estruturas de controle de fluxo

Expressões e operadores aritméticos

Operador	Símbolo
Soma	+
Subtração	—
Multiplicação	*
Divisão	/
Resto (mod)	%

Estruturas de controle de fluxo

Operadores lógicos

Operador	Símbolo
Igual a	==
Diferente de	!=
Maior que	>
Menor que	<
Maior ou igual a	>=
Menor ou igual a	<=

Estruturas de controle de fluxo

Controle de fluxo

```
fn main() {  
    let numero = 3;  
  
    if numero < 5 {  
        println!("condição verdadeira!");  
    } else {  
        println!("condição falsa...");  
    }  
}
```

Estruturas de controle de fluxo

Controle de fluxo

```
fn main() {  
    let numero = 3;  
  
    if numero < 5 {  
        println!("condição verdadeira!");  
    } else if numero == 3 {  
        println!("condição parcial...");  
    } else {  
        println!("Número qualquer!");  
    }  
}
```

Agenda

1. Ambiente de desenvolvimento
2. Linguagem Rust
3. Variáveis e tipagem
4. Estruturas de controle de fluxo
- 5. Atividade**

Atividade

1. Crie um programa em Rust que recebe como entrada a idade de um usuário e, se for maior de idade, mostre na tela que ele é maior de idade. Caso contrário, mostre que ele é menor de idade.
2. Crie um programa para autenticar um usuário. Solicite o nome de usuário e senha e, em seguida, verifique se o nome de usuário é `admin` e a senha é `@dm1n`. Caso a senha ou o usuário não corresponderem, mostre na tela que o usuário ou a senha estão incorretos. Caso contrário, mostre que o usuário foi autenticado.
3. **[DESAFIO]** Crie um programa em Rust que faça o cálculo dos números pares da sequência de Fibonacci. Implemente o algoritmo de maneira **iterativa** (sem recursividade).